# Как сдать результаты

Единый формат, чтобы организаторы могли сопоставить оценки команд.

## 10 контрольных чатов · 1 час

Через 23 часа после старта вы получите контрольный набор. До стоп-кода, через 24 часа после старта, нужно обработать все 10 чатов и загрузить результаты в Git.

### 01 Папки в корне репозитория

Создайте две папки. В каждой разместите один файл **results.json** с результатами соответствующего набора.

```
results/
  dev/
    results.json
  prod/
    results.json
```

**results/dev/results.json** - результаты по DEV: 60 чатов и 5 звонков.

**results/prod/results.json** - результаты по 10 контрольным чатам. Доступ к нему и способ получения организаторы сообщат за час до стоп-кода.

### 02 Что должно успеть попасть в Git

**Получите набор.** Используйте выданный организаторами API и сохраните исходные conversation_id.

**Оцените диалоги.** Выгрузите результаты работы своей системы по всем восьми критериям чек-листа.

**Проверьте JSON.** Одна запись на каждый диалог, без пропусков и повторяющихся ID.

**Сделайте commit и push.** Оба файла должны быть доступны в открытом репозитории по ссылке, заранее переданной организаторам, до стоп-кода.

Час включает получение данных, оценку, проверку и отправку файлов. Локального сохранения или commit без push недостаточно.

# Как выглядит JSON

Один объект на файл. В массиве results - все диалоги выбранного набора.


```
{
  "schema_version": "1.0",
  "team_name": "Название команды",
  "model": {"mode": "local", "name": "Название модели"},
  "results": [
    {
      "conversation_id": "demo-001",
      "status": "ok",
      "criteria": [
        {"criterion_id": "contact", "value": true,
         "rationale": "Оператор поздоровался и представился.",
         "evidence_turns": [4]},
        {"criterion_id": "discovery", "value": null,
         "rationale": "Клиент сразу ясно изложил вопрос.",
         "evidence_turns": [1]},
        {"criterion_id": "identification", "value": null,
         "rationale": "Общий вопрос не требует идентификации.",
         "evidence_turns": [1]},
        {"criterion_id": "consultation", "value": 15,
         "rationale": "Основной вопрос решён, упущена деталь.",
         "evidence_turns": [5, 6]},
        {"criterion_id": "empathy", "value": true,
         "rationale": "Вежливо признал неудобство клиента.",
         "evidence_turns": [4, 5]},
        {"criterion_id": "dialogue", "value": 7,
         "rationale": "Есть повтор, но ход диалога понятен.",
         "evidence_turns": [5, 6]},
        {"criterion_id": "closure", "value": true,
         "rationale": "Предложил дальнейшую помощь и попрощался.",
         "evidence_turns": [8]},
        {"criterion_id": "critical", "value": false,
         "rationale": "Грубых нарушений в репликах нет.",
         "evidence_turns": [4, 5, 6, 8]}
      ],
      "score": {
        "final_score": 85.38,
        "critical_flag": false
      }
    }
  ]
}
```


Это вымышленный пример структуры, не ответ на диалог из набора. Вместо demo-001 подставьте исходный ID. Добавьте остальные объекты в results.

Для контрольного файла структура та же. Набор определяется папкой dev или prod. Не меняйте названия полей, коды критериев и типы значений.

# Правила заполнения

Используйте действующий чек-лист. Оценивайте работу оператора.

| criterion_id / критерий | value | Вес |
|---|---|---|
| contact · Установление контакта | true / false | 10 |
| discovery · Выявление потребности | true / false / null | 15 |
| identification · Идентификация | true / false / null | 20 |
| consultation · Корректность и полнота | 0, 5, 10, 15, 20 / null | 20 |
| empathy · Эмпатия | true / false | 15 |
| dialogue · Ведение, скорость, грамотность | 0, 4, 7, 10 | 15 |
| closure · Завершение | true / false / null | 5 |
| critical · Грубые ошибки | true / false | - |

**value:** true = «Да», false = «Нет», null = N/A. Число - оценка по шкале критерия, до учёта веса. Например, dialogue = 7 означает 7 из 10, при весе 15. Для status = "ok" нужны все 8 критериев, каждый ровно один раз.

**rationale и evidence_turns:** кратко объясните оценку и укажите номера подтверждающих реплик. Нумерация с 1 по исходному массиву транскрипта, включая клиента, бота и события. Не перенумеровывайте только реплики оператора.

**final_score:** итог от 0 до 100, округлённый до двух знаков. При N/A исключите критерий и пропорционально перераспределите его вес между применимыми критериями. N/A не означает ноль или ошибку обработки.

**critical_flag:** совпадает с value критерия critical. true означает выявленное грубое нарушение. Этот флаг передаётся отдельно и не обнуляет итог автоматически.

**model:** укажите фактически использованную модель в name; mode - local, external_api или hybrid. Если моделей несколько, перечислите их названия в name.

### Если оценить диалог не удалось

Сохраните его ID, status = "error", пустой criteria, score = null и причину в error. Не скрывайте такой диалог и не подменяйте ошибку нулевой оценкой.

```
{"conversation_id": "demo-002", "status": "error",
 "criteria": [], "score": null,
 "error": "Не удалось получить ответ модели."}
```

**Перед push:** JSON в UTF-8, без комментариев и лишних запятых. ID точно совпадают с API. В prod - ровно 10 записей, включая ошибки. Результаты доступны в репозитории до стоп-кода.